

Level 4. Fleets, cloud and scale

Levels 1 to 3 designed one robot. This level asks what changes when there are two hundred of them. The short answer: the robot stays the same, and a new system grows up around it. That new system has its own boxes, its own arrows, and its own trade-offs, and most of them are about the network, which is the one arrow you cannot trust.

Rough time: two evenings. No code. You need level 1 (boxes, arrows, trade-offs) and the idea from level 3 that every arrow has a latency, a throughput, and a failure mode.

4.1 From one robot to many

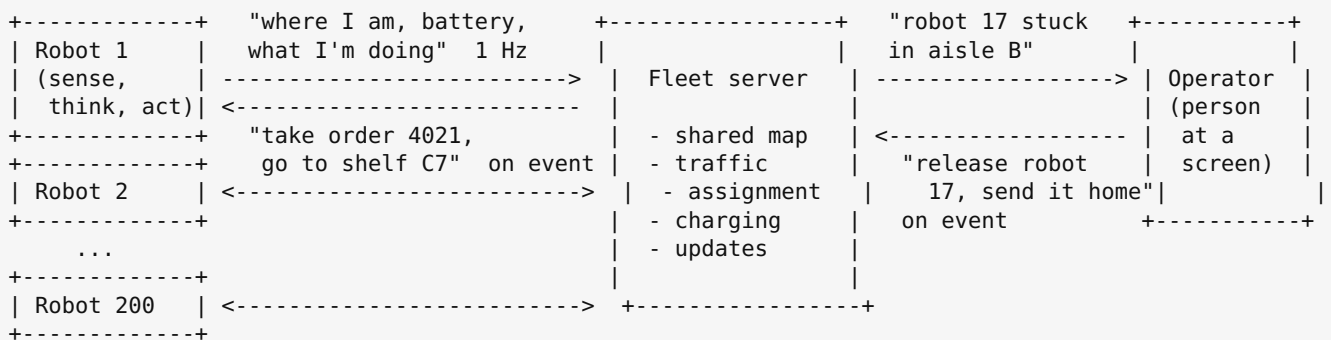
Picture a warehouse the size of two football pitches. Shelves in long rows. One robot drives between the shelves, picks up a tote (a plastic crate of goods), carries it to a packing station, and comes back. You designed that robot in level 2. It has a map, a planner, a controller, a battery.

Now put two hundred of them on the same floor.

Nothing about a single robot's sense, think, act loop changes. But five new problems appear that did not exist with one robot:

1. **Shared map.** Two hundred robots must agree on where the shelves are. If robot 17 sees a fallen tote in aisle B and robot 90 does not know, robot 90 will plan straight through it. The map stops being one robot's private file and becomes something the whole fleet shares.
2. **Traffic.** Two robots that both want the same corridor at the same time will block each other, or worse, deadlock: each waiting for the other to move. With one robot, a path was a path. With two hundred, a path is a claim on space and time.
3. **Task assignment.** Five hundred orders arrive in an hour. Which robot takes which order? The nearest one? The one with the fullest battery? The one that is already going that way?
4. **Charging.** Each robot runs for about eight hours and charges for one. At any moment, roughly one in nine robots is charging. For two hundred robots that is about twenty-two robots on chargers at once, so you need at least that many chargers, plus spare, and a rule for who charges when so they do not all go at the same time.
5. **Updates.** A bug fix that took a USB stick and five minutes on one robot now has to reach two hundred robots, safely, without stopping the warehouse.

Each of these needs a box that is not on any single robot. That box is called the **fleet server** (or fleet manager). It is a computer, usually in the warehouse's server room or in the cloud, that sees all robots at once. Above it sits the **operator**: a person watching a screen who steps in when something goes wrong.



Read the arrows. Up from each robot: a small summary, about once a second. Down to each robot: a task or an instruction, only when something changes. Up to the operator: exceptions, not everything. Down from the operator: decisions, rarely.

The main idea of this level is in that picture. The robot is still a full, self-contained sense, think, act loop. The fleet server does not drive the robot. It tells the robot *what* to do, and the robot works out *how*. Keep that split in your head; every section below is a consequence of it.

4.2 What runs on the robot and what runs in the cloud

Cloud just means computers somewhere else that you reach over a network. It might be a rack in the warehouse office (people say "on-premises" or "on-prem" for that) or a rented machine in a data centre. For design purposes the difference is small. The thing that matters is that there is a network between the robot and the cloud, and networks fail.

Here is the split, with the reason for each row.

Job	Where	Why
Motor control, balance, emergency stop	Robot, always	Needed to stop safely. Must work with the cable unplugged.
Obstacle detection and local avoidance	Robot, always	A person steps out at 1 m. There is no time for a network round trip.
Localisation (where am I on the map)	Robot	Needs sensor data at full rate; sending it away is too much data (see 4.3).
Local path planning (next 10 m)	Robot	Must react to what the robot sees now.
Map building	Either	On the robot when exploring alone; in the cloud when merging 200 robots' views into one map.
Global route across the warehouse	Cloud, usually	The server knows where every other robot is, so it can avoid traffic. The robot can fall back to its own A* if the server is silent.
Task assignment (which robot takes which order)	Cloud	Needs to see all robots and all orders at once. One robot cannot.
Charging schedule	Cloud	Same reason: it is a fleet-wide decision.
Analytics (which aisle causes the most stops)	Cloud	Needs weeks of data from all robots. Not time critical.
Software updates	Cloud sends, robot installs	The robot must be able to say "not now, I am carrying a tote".

The rule that generates the whole table:

Anything the robot needs in order to stop safely must not depend on the network.

State it as a trade-off, in the level-1 sentence: local avoidance runs on the robot because a safe stop matters more than having the cleverest possible planner. Task assignment runs in the cloud because seeing the whole fleet matters more than working when the network is down, and a robot with no task simply waits, which is safe.

A useful test for any box: unplug the network cable and ask what the robot does in the next two seconds. If the answer is "keeps driving on stale commands", the box is in the wrong place.

4.3 Communication: the arrow you cannot trust

Wi-Fi, 4G and 5G in plain words

Wi-Fi is a short-range radio link to a box on the wall called an **access point**. Range is tens of metres indoors. A warehouse has many access points, and a moving robot hops from one to the next. Each hop can drop the connection for a fraction of a second to several seconds. Wi-Fi is what you use indoors because you own the access points and pay nothing per byte.

4G and **5G** are the mobile phone networks. Range is kilometres, you do not own the towers, and you pay per gigabyte. 4G gives a robot on a pavement a few megabits per second uploading, with a round trip (send a message and get a reply) of roughly 50 to 100 milliseconds on a good day. 5G is faster and can have a shorter round trip, but coverage on UK streets is patchy, so you design for 4G and treat 5G as a bonus. Outdoors you use 4G or 5G because you cannot put an access point on every lamp post.

Both kinds of link have two properties you design around:

- **Bandwidth**: how many bits per second you can push through. Shared with everyone else on the same access point or tower.
- **Dropouts**: periods of seconds to minutes where nothing gets through at all. Behind a metal shelf, inside a lift, under a railway bridge. These are normal, not faults.

Why you send summaries, not raw sensor data

Do the arithmetic. Level 1 said one colour camera at 640 by 480 pixels, 30 frames a second, is about 28 megabytes a second. In bits, that is:

$$640 \times 480 \text{ pixels} \times 3 \text{ bytes} \times 30 \text{ frames/s} = 27.6 \text{ MB/s} = 221 \text{ Mbit/s}$$

A single warehouse Wi-Fi access point shares about 100 Mbit/s between everyone on it. One raw camera stream is already twice that. Now two hundred robots:

$$200 \times 221 \text{ Mbit/s} = 44,000 \text{ Mbit/s} = 44 \text{ Gbit/s}$$

That is not a Wi-Fi problem, that is a "buy a data centre" problem. And that is one camera per robot, no lidar.

Compress the video (the way a video call does) and each stream drops to about 2 Mbit/s. Two hundred robots then need 400 Mbit/s. Still four access points completely saturated, just for video nobody is watching.

Now send a summary instead. Each robot sends, once a second: its position (x, y, heading), a timestamp, battery percentage, and a state code ("carrying tote", "charging", "stuck"). With message overhead, call it 200 bytes.

$$200 \text{ robots} \times 200 \text{ bytes} \times 1 \text{ per second} = 40 \text{ kB/s} = 320 \text{ kbit/s}$$

That is 0.3 percent of one access point. The ratio between one robot's raw camera and its summary is about 138,000 to one. This is why the fleet server never sees images in normal operation. It sees positions and states. The robot does perception; it sends conclusions.

The exception is when a person needs to see. In 4.7 an operator will ask for one robot's video, and one compressed stream at 2 Mbit/s is fine. The design rule: **video on demand, for one robot at a time; summaries always, for all of them.**

Message queues: the MQTT idea

With two hundred robots sending and one server receiving, you do not want two hundred direct cables into one box. You put a **broker** in the middle. A broker is a program that receives messages on named channels, called **topics**, and hands them to whoever has asked for that topic. The robot **publishes** to a topic; the server **subscribes** to it.

```
Robot 17 --"fleet/17/status" 1 Hz--> +-----+ --all "fleet+/status"--> Fleet server
Robot 18 --"fleet/18/status" 1 Hz--> | Broker | --"fleet/17/status"----> Operator screen
Fleet srv --"fleet/17/task" on event-> | (MQTT) | --"fleet/17/task"-----> Robot 17
                                     +-----+
```

This is the same publish and subscribe idea as a ROS 2 topic (level 2), and it exists with or without ROS. The most common broker protocol for robots talking to a server over unreliable links is **MQTT** (an old name, originally "Message Queuing Telemetry Transport"; today just treat it as a name). MQTT is popular for three reasons that map directly onto the problems above:

1. Messages are tiny, a few bytes of overhead, so summaries stay small.
2. Each message can ask for a delivery level: "fire and forget" for status (a lost one is replaced a second later) or "deliver at least once" for a task (must not be lost).
3. The broker can hold a **last will**: a message it sends on the robot's behalf if the robot goes silent. That is how the server learns robot 17 dropped off the network without robot 17 saying anything.

The trade-off: a broker is one more box that can fail. You accept it because decoupling two hundred senders from the server matters more than one extra box, and you run two brokers so one can die.

Store-and-forward when offline

A robot behind a metal rack loses Wi-Fi for ten minutes. What happens to the data it wanted to send?

Store-and-forward means: write it to local disk, keep working, send it when the link comes back. At 1 kB/s of logs and status, ten minutes is 600 kB. A day is 86 MB. A cheap flash drive holds weeks. So the robot never throws data away just because the network is down.

The design decision inside store-and-forward is *what to send first when the link returns*. The current status, so the server knows the robot is alive now. Then anything safety related (a near-miss). Then the backlog of routine logs, slowly, so the catch-up does not starve the robots that are online. That is a priority queue on the robot, and [tutorial/05-heaps-trees-design.md](#) is where you build one.

Meanwhile, what does the robot *do* while offline? It finishes its current task, which it already has. If it finishes and has no new task, it waits or goes to a safe spot. It never guesses. The rule from 4.2 again: offline must be safe, and "waiting" is safe.

4.4 Fleet-level planning

Traffic: a path is a claim on space and time

One robot plans a path as a list of cells. Two robots must plan paths that do not put both of them in the same cell at the same time. That adds time as a dimension. A path becomes a list of (cell, time step).

Here is a five by five patch of warehouse. Robot X wants to cross left to right along row 2. Robot Y wants to cross top to bottom along column 2.

```
      col 0  1  2  3  4
      +-----+
row 0  |  |  | Y |  |      Y: starts (2,0), goes down column 2
      +-----+
      |  |  |  |  |
      +-----+
```

```

row 1 | | | . | | |
      +---+---+---+---+
row 2 | X | . | ? | . | . |      X: starts (0,2), goes right along row 2
      +---+---+---+---+
row 3 | | | . | | |
      +---+---+---+---+
row 4 | | | . | | |
      +---+---+---+---+

```

Both move one cell per time step. X is at column t at time t . Y is at row t at time t . At $t = 2$, X is at (2,2) and Y is at (2,2). Collision. A single-robot planner never sees this, because each robot's map shows the cell as empty.

Three simple ways to solve it. Real warehouses use a mix.

1. Reserve cells with time. The fleet server keeps a **reservation table**: a set of (cell, time) pairs that are taken. X plans first and reserves (0,2) at $t=0$, (1,2) at $t=1$, (2,2) at $t=2$, and so on. Y plans second, treating any reserved (cell, time) as a wall. Y finds that (2,2) at $t=2$ is taken, so it waits one step at the start: (2,0) at $t=0$ and $t=1$, then (2,1) at $t=2$, (2,2) at $t=3$. No conflict, one step lost. Checked: with Y delayed by one step, no cell is shared at any time, and there is no "swap" (X and Y passing through each other).

This is *A on a grid with a third coordinate, time*. The A you build in Part C of tutorial/03-grids-bfs-dfs.md is exactly the planner box here; the only change is that the "is this cell free" check also asks "at this time step". The cost is that the reservation table grows with robots times path length, and a robot that is late breaks everyone's reservations behind it. You accept it because in a warehouse the floor is yours, the robots are punctual, and near-perfect use of narrow aisles matters more than simplicity.

2. Priority order. Number the robots. When two conflict, the lower number goes and the higher number waits. No table needed; each robot only needs to know the plans of robots with lower numbers. Simple and robust. The cost is that robot 200 can be starved forever in a busy crossing. You accept it in small fleets where simplicity matters more than fairness. Priority can be smarter than a fixed number: "robot carrying a tote beats an empty one", "robot with lower battery beats one with more".

3. One-way lanes. Redraw the map so that aisles are one-way, like a road system. Then head-on conflicts cannot happen at all, and only crossings need rules. This is the cheapest solution because it removes the problem instead of solving it. The cost is longer paths: a robot may drive round three sides of a block to reach a shelf that was two metres behind it. You accept it when the fleet is large and the aisles are narrow, because never deadlocking matters more than the shortest route.

One-way layout of the same patch:

```

          col 0  1  2  3  4
      +---+---+---+---+
row 0 | > | > | > | > | v |    > east only
      +---+---+---+---+
row 1 | ^ |   |   |   | v |    v south only
      +---+---+---+---+
row 2 | ^ |   |   |   | v |    ^ north only
      +---+---+---+---+
row 3 | ^ |   |   |   | v |    < west only
      +---+---+---+---+
row 4 | ^ | < | < | < | < |
      +---+---+---+---+

```

Under any of these, the global route is a **graph** problem: junctions are nodes, lane segments are edges, and the edge weight is travel time, which the server can raise for a congested lane. That is Dijkstra from tutorial/04-graphs-dijkstra.md, run on the server, with weights that change every few seconds as traffic changes.

The split between server and robot on planning is then:

Level	Where	Input	Output	How often
Route (which aisles)	Server	fleet positions, one-way map, congestion	list of junctions	on new task, and re-plan if a lane blocks
Path (which cells, when)	Server or robot	route, reservation table	list of (cell, time)	on new route
Local (next 2 m)	Robot only	lidar, path	speed and steer	20 Hz

Task assignment: which robot takes which order

Orders arrive. Robots are free. Match them. Two common ways:

Nearest-first. Take orders in the order they arrive. For each one, give it to the nearest free robot. Simple, fast, easy to explain.

Auction. When an order appears, announce it to all robots. Each robot replies with a **bid**: its cost to do this order (distance, or time, including any work it already has). The lowest bid wins. Robots that are far away or busy bid high and lose. This is the same matching problem, but the robots do the costing, so it scales and keeps working if the server's picture of a robot is slightly out of date.

Both are **greedy**: they make the best choice for one order at a time and never go back. A worked example on a straight aisle shows why that can hurt.

Position: 0 1 2 3 4 5 6 7 8 9 10 11 12 13
 R1 A R2 B R3 C

R1 at 0, R2 at 5, R3 at 12. Orders arrive in the order A (4), B (6), C (13).
 Cost = distance along the aisle.

Nearest-first, orders in arrival order:

Order	Free robots and distances	Chosen	Cost
A at 4	R1: 4, R2: 1, R3: 8	R2	1
B at 6	R1: 6, R3: 6	R1	6
C at 13	R3: 1	R3	1
		Total	8

Now try all six ways of pairing three robots with three orders and add up the distances. The best is R1 to A (4), R2 to B (1), R3 to C (1): **total 6**. Nearest-first gave 8. It grabbed R2 for A because R2 was closest, and left R1 with a long walk to B. Greedy is a third worse here, and in a real warehouse the gap can be bigger.

The trade-off: the best matching of N robots to N orders can be found exactly (the method is called the Hungarian algorithm; you do not need its details), but it costs roughly N cubed work and needs all orders known up front. Nearest-first costs almost nothing and works one order at a time as they arrive. Most fleets run nearest-first or an auction because orders arrive continuously and a decision in ten milliseconds matters more than a perfect answer. Then they add a fix: every few seconds, re-run assignment on all *unstarted* orders and swap if it helps. That recovers most of the gap.

Note that a plain "lowest bid wins, one order at a time" auction gives the same total of 8 on this example, because it is the same greedy choice made by the robots instead of the server. The auction's advantage is not a better answer; it is that the robots compute their own costs, so a robot with a low battery or a half-finished job can bid high on its own without the server needing to model it.

4.5 Maps and data

Who owns the map

With one robot, the map is a file on the robot. With two hundred, the fleet server owns the **master map** and each robot holds a copy. The map is versioned: every change gets a new number, and every robot reports which version it is running.

```

Robot 41  --"new obstacle at (12,7), map v83"---> +-----+
                                           | Fleet server | --"map v84" once--> Robot 1
                                           | master map   | --"map v84" once--> Robot 2
                                           | v83 -> v84   | ...
Operator  --"confirm obstacle"-----> +-----+ --"map v84" once--> Robot 200
                                           on event

```

Robots do not edit the master map directly. They *propose* changes ("I saw a pallet here that is not on the map") and the server merges proposals. A single robot's false detection must not delete a corridor for two hundred robots. In practice: a change seen by one robot is marked "provisional" and routes avoid it; a change seen by three robots, or confirmed by the operator, becomes permanent.

How updates get to robots: the server publishes "map v84 available". Each robot fetches the difference from its version (only the changed cells, a few kilobytes) rather than the whole map (megabytes). It swaps to the new version at a safe moment, between tasks, never in the middle of a narrow aisle. A robot on an old map version is allowed to keep working, but the server plans its routes on the *older* of the two versions so both agree on what exists.

Versioning is the whole trick. Without a version number you cannot tell whether robot 90's "there is no obstacle at (12,7)" is a disagreement or just old news.

What data to collect from the fleet, and why

You have two hundred robots driving eight hours a day. That is 1,600 robot-hours a day of experience. It is the most valuable thing the fleet produces after the orders themselves, because it tells you where the design is wrong. Collect, at a minimum:

Data	Why	Size
Failures: what stopped, where, when, last 30 s of state	Find the top three causes and fix them.	small; a few kB per event
Near-misses: emergency stops that triggered, obstacles closer than the safety margin	Failures you got away with. Same causes, ten times more samples.	a few kB per event, plus a short video clip on request
Battery curves: charge level against time, for each robot	Spot a robot whose cell is dying before it strands a tote in an aisle. Plan charger count.	1 byte per second per robot
Task times: assigned, started, finished	Measure whether assignment changes help. Feed the analytics.	tiny
Map proposals accepted and rejected	Tune the perception threshold.	tiny

Notice what is not on the list: continuous video. It is too big (4.3) and mostly useless. You keep a rolling buffer of the last 30 seconds on the robot and upload it only when an event happens. That is store-and-forward with a trigger.

Privacy of camera data on pavements

A warehouse camera sees shelves. A pavement delivery robot's camera sees people's faces, car number plates, and the inside of front windows. In the UK that is personal data, and the rules (the UK General Data Protection Regulation, GDPR) apply. You do not need legal detail, but you need design rules:

- Process on the robot, send conclusions. "Person 2 m ahead" leaves the robot. The image does not. This is the same rule as bandwidth, for a different reason.
- If an image must leave the robot (a crash, a remote operator), blur faces and plates on the robot before sending, and delete the clip after a fixed time unless it is part of an incident.
- Never keep video "just in case". Storage you do not have cannot leak.

The trade-off: blurring costs compute on the robot and makes the operator's picture worse. You accept it because not becoming a walking surveillance camera matters more than a crisp teleoperation view.

4.6 Updates and configuration

Over-the-air updates

An **over-the-air (OTA) update** is new software sent to the robot over the network and installed without anyone touching the robot. It is the only practical way to update two hundred robots, and it is also the fastest way to break all two hundred at once.

A full software image for a robot computer is around 500 MB. At 20 Mbit/s per robot that is about three and a half minutes each. Two hundred robots is 100 GB; on one 100 Mbit/s access point, over two hours if nothing else used the network. So updates download in the background, slowly, while robots work, and only *install* when the robot is docked. Downloading and installing are separate steps with separate commands.

The robot always keeps two copies of its software: the one running and the one being installed. The install writes the new copy, checks it, then flips a switch to boot from it. If the new copy fails to start, the switch flips back. This is called an A/B partition scheme and it is what makes the next two ideas possible.

Staged rollout: 1, then 10, then all

Never send an update to the whole fleet at once. Stage it:

Stage 1:	1 robot	soak 1 day	Does it boot? Does it drive? Any new errors?
	v (only if stage 1 is clean)		
Stage 2:	10 robots	soak 1 day	Same questions, across different aisles and shifts.
	v (only if stage 2 is clean)		
Stage 3:	all 200	over a shift	Rolling: never more than 10 percent installing at once.

"Clean" is defined before you start: no new failure types, task time not worse than 2 percent, no near-miss increase. If any number is bad, you stop and roll back the robots that have it.

The trade-off: a staged rollout takes three days instead of an hour. You accept it because one broken robot for a day costs far less than two hundred broken robots for an hour, and because you can never test a warehouse floor as well as the warehouse floor tests itself.

Rollback

Rollback means putting the previous version back. With A/B partitions it is one command: flip the switch, reboot. Two rules:

- Every update must be rollback-safe. If version 84 changes the format of the map file, version 83 must still be able to read what 84 wrote, or the rollback breaks worse than the update did.
- Rollback is automatic when the robot itself detects it is unhealthy after an update (fails to start, cannot reach the broker, watchdog fires). It does not wait for a person.

Config versus code

Code is the program. **Configuration** (config) is the numbers the program reads at startup: maximum speed, safety margin, which map, which broker address. Keep them separate.

Changing config is cheap: a small file, no reboot, applied at the next safe moment. Changing code is expensive: staged rollout, days. So design so that anything an operator might need to change on a Tuesday afternoon ("slow the robots in aisle F, there is a spill") is config, not code. And version config too: "robot 17 runs code v84 with config v212" is a complete description; "robot 17 has the new settings" is not.

The trade-off: too much in config, and nobody knows what the robot actually does without reading a hundred numbers. A good middle: a handful of operator-facing knobs in config, the rest in code with tests.

4.7 Monitoring and operations

What an operator sees

The operator is the person on the human end of the arrow in 4.1. Their screen is the fleet server's view drawn for a human. A useful one shows, top to bottom:

```
+-----+
| FLEET: 200 robots   Working 171   Charging 21   Idle 5   NEEDS HELP 3   |
+-----+
| ALERTS (newest first)                                     |
|  !! 14:02 Robot 17   stuck 90 s, aisle B row 12   [view] [assist] |
|  ! 13:58 Robot 103  battery 9 %, 400 m from charger |
|  ! 13:51 Robot 66   map proposal: obstacle at (12,7), unconfirmed [ok] |
+-----+
| MAP                                                         | FLEET HEALTH (last hour) |
| . . . . . | Orders/hour      480   (target 500) |
| . 3 . . . . 9 . . . . 41 . . | Avg task time    3.1 min |
| . . . . 17! . . . . . | E-stops          2      |
| . . 88 . . . 12 . . . 55 . . | Wi-Fi dropouts   14     |
+-----+
```

The design rule for a dashboard is the same as for the network: **exceptions, not everything**. The operator cannot watch two hundred dots. They watch three alerts and a handful of numbers that drift when something is wrong across the fleet (orders per hour falling, dropouts rising).

Alerts

An alert is a rule on the server that turns data into a line on that screen. Three rules make alerts useful rather than noise:

1. **Every alert says what to do.** "Robot 17 stuck" is a fact. "Robot 17 stuck 90 s, aisle B row 12, [assist]" is an alert.

2. **Alerts have levels, and the levels mean something.** Red: a person acts now. Amber: a person acts this shift. Grey: logged, look at it weekly. If everything is red, nothing is.

3. **An alert clears itself.** When robot 17 moves, the line goes. The operator never tidies.

The trade-off: a threshold too low floods the screen; too high and a real stuck robot waits ten minutes. You tune with data from 4.5, and you start with thresholds that fire rarely, because an operator who trusts the screen matters more than catching every hiccup.

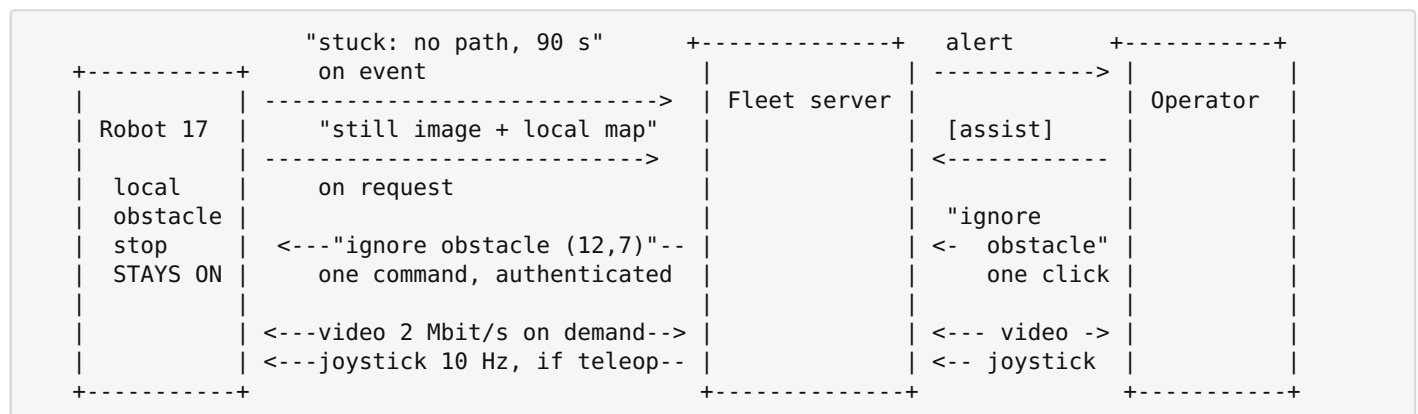
Remote assistance and teleoperation

A robot is stuck. Its planner has no path: a pallet blocks the aisle, or a person parked a trolley across it, or it thinks there is an obstacle that is not there. Options in order of cost:

1. **Remote nudge.** The operator looks at the robot's map view and a still image, and clicks "ignore that obstacle" or "reverse 1 m and re-plan". The robot does the driving. One picture, a few bytes back. This solves most cases.
2. **Teleoperation.** The operator drives the robot with a joystick over a live video stream. This is the expensive case: one 2 Mbit/s stream and a person's full attention.
3. **Send a person.** Someone walks to aisle B.

Teleoperation has a latency limit. Video from the robot's camera to the operator's screen (called glass-to-glass) over 4G is around 300 milliseconds. At 1.5 m/s, the robot moves 0.45 m before the operator sees anything. At 0.5 m/s it is 0.15 m. So teleoperation is done at a crawl, and the robot's own obstacle stop stays on. The operator is steering, not overriding safety. Rule from 4.2, again.

The human-in-the-loop path, drawn



Every command on the path is logged with who sent it and when. Every command goes through the fleet server, never operator to robot directly, so there is one place to check permissions (4.9) and one log.

Expected load: if each robot needs help twice per eight-hour shift, for three minutes each, that is 1.25 percent of the time. For fifty robots, on average less than one is stuck at a time. One operator per fifty robots is a common starting ratio and this arithmetic is why.

4.8 Safety cases and standards, in plain words

What a safety case is

A **safety case** is a written argument that the robot is safe enough to use, backed by evidence. It has three parts: what could go wrong, what you did about each thing, and how you know it worked. It is a document, not a test. Regulators, insurers, and a warehouse owner's safety officer read it. You write it during design, not after, because the cheapest fix for a hazard is a design that cannot produce it.

The main computer sends a heartbeat. If it stops, the safety controller cuts power. That is the **watchdog** from level 3, in hardware. The main computer can *request* a stop; it can never *prevent* one. And the fleet server cannot reach the safety controller at all, which is the point of 4.2's rule made physical.

"Safety-rated" means the part has been tested and certified to a standard to fail in a known, safe way, and its failure rate has been measured. It costs more. You accept it because a certified stop that works when the main computer is on fire matters more than the price of a second small board.

Three standards by name

You will see these names in job adverts and design reviews. One line each; no legal detail.

- **ISO 13482**: the safety standard for personal care robots, meaning robots that work close to ordinary people, such as a delivery or assistance robot on a pavement or in a home.
- **ISO 3691-4**: the safety standard for driverless industrial trucks, which covers warehouse robots and automated guided vehicles moving goods around a site.
- **ISO 26262**: the functional safety standard for road vehicles' electronics, which is what autonomous car software and hardware is assessed against.

A standard tells you what evidence a safety case needs. It does not design the robot for you.

4.9 Security basics

Safety is about accidents. **Security** is about people who want the robot to do the wrong thing on purpose. Three basics for a fleet.

Signed updates. Every software image is signed with a private key that only your release process has. The robot checks the signature with the matching public key before installing. An image without a valid signature is refused, even if it came from the right server, because "the right server" is a network address, and network addresses can be faked.

Authenticated commands. Every command that reaches a robot, "go to C7", "ignore obstacle", "install v84", carries proof of who sent it, and the robot checks that this sender is allowed to send this kind of command. An operator can nudge a stuck robot; only the release process can install software; nobody can turn the safety controller off, because there is no command for it. The broker in 4.3 enforces this too: a robot may publish only to its own status topic and subscribe only to its own task topic. Robot 17 cannot send a task to robot 18.

Why an open emergency stop API is dangerous. It seems harmless, even good, to have a network command that stops every robot. Think about who else can send it. A stop command that anyone on the Wi-Fi can send is a button that halts your warehouse. Two hundred robots stopping at once in narrow aisles, some mid-turn, some on ramps, with totes in the air, is itself a hazard, and it is also a way for one person with a laptop to shut a business down. So:

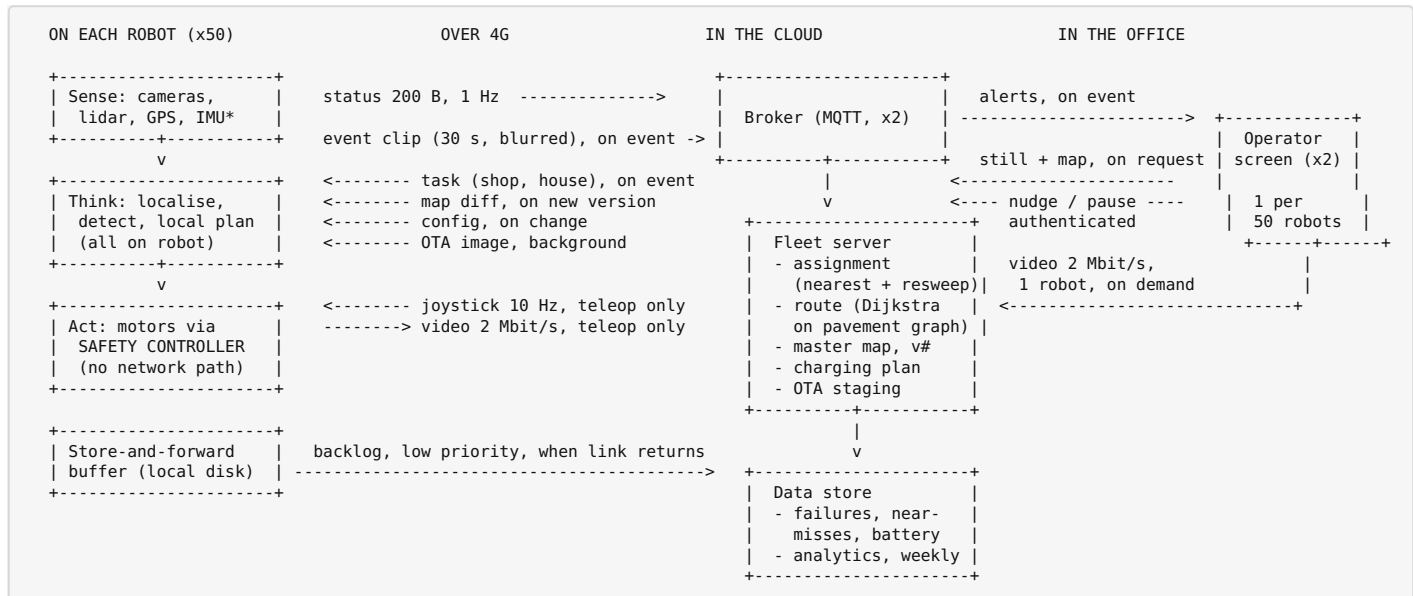
- The physical e-stop is a button, wired to the safety controller, not on the network at all.
- A fleet-wide "pause" command exists, authenticated, limited to two named operator accounts, logged, and it makes robots *finish their current motion and stop in a safe spot*, not cut power.
- Robots reject a pause from anyone else, and report the attempt as an alert.

The trade-off: authentication adds a little latency and a lot of key management. You accept it because a fleet that only its owners can command matters more than a slightly simpler message format.

4.10 Worked example: a 50-robot city delivery fleet

Level 1 asked you to draw a pavement delivery robot. Now scale it. The job: fifty robots delivering from twenty shops to homes across a UK town, on pavements, over 4G, with two operators in an office. Let us design the fleet layer from the ideas above.

The full picture



(*IMU: inertial measurement unit, a chip that senses acceleration and turning rate.)

The numbers behind it, checked:

- **Status:** 50 robots at 200 bytes, once a second, is 80 kbit/s for the whole fleet. Trivial.
- **Per robot on 4G:** 1 kB/s of status and logs continuously is 86 MB a day, about 2.6 GB a month. Fits a cheap data plan. One compressed camera stream at 1 Mbit/s continuously would be 10.8 GB a day, 324 GB a month, per robot. That is the whole argument for event clips.
- **Teleoperation:** if all fifty streamed at once it would be 100 Mbit/s, but they never do. At two stuck events per shift of three minutes each, under one robot on average needs help at any moment. Two operators is ample; one is tight when two robots get stuck together.
- **Charging:** eight hours running, one charging means about one in nine charging at once, roughly six of fifty. Put eight charging bays in the depot and stagger the schedule so the lunchtime peak has every robot out.
- **Teleop latency:** 300 ms glass-to-glass, so teleop is capped at 0.5 m/s (the robot moves 0.15 m before the operator sees it), and the safety lidar stop stays active.

Interface table

Arrow	What travels	How often	If it stops
Robot → Broker: status	position, heading, battery, state code, timestamp, map version, code version (about 200 B)	1 Hz	broker's last-will marks robot "silent"; alert after 30 s
Robot → Broker: event clip	30 s of blurred video plus state log, on a failure or near-miss	on event	buffered on robot, sent later
Robot → Data store: backlog	logs since last contact	when link returns, low priority	stays on disk; weeks of space

Arrow	What travels	How often	If it stops
Server → Robot: task	shop id, house address, route as list of junctions, deadline	on assignment, at least once delivery	robot finishes current task, then waits at a safe spot
Server → Robot: map diff	changed cells, from robot's version to newest	on new version	robot keeps old version; server routes it on the old version
Server → Robot: config	speed cap, safety margin, zones	on change	robot keeps last config
Server → Robot: OTA image	signed image, background download	per staged rollout	download resumes; install only when docked
Operator → Server → Robot: nudge	"ignore obstacle", "reverse and re-plan", "pause"; authenticated, logged	on click	robot stays stopped; still safe
Operator ↔ Robot: teleop	video 2 Mbit/s down, joystick 10 Hz up	during assist only	robot stops within 200 ms of last joystick message (watchdog)
Sensors → Safety controller → Motors	obstacle-in-zone, bumper, e-stop, heartbeat	continuous	power cut; nothing on the network can prevent this

Three trade-offs

- 1. Assignment is nearest-first with a re-sweep every ten seconds, not an exact matching.**
Because orders arrive all day and a robot must be moving within a second of an order appearing, responsiveness matters more than the last five percent of distance saved. The re-sweep on unstarted orders recovers most of the gap, as in the 8 versus 6 example.
- 2. Perception and every safety decision stay on the robot; the cloud only sees conclusions.**
Because a 4G dropout under a railway bridge must leave the robot safe and still moving, and because faces on pavements must not leave the robot. This costs a more powerful computer on every robot, fifty times over. Safe-when-offline and privacy matter more than a cheaper robot.
- 3. The fleet-wide pause makes robots finish their motion and pull in, rather than cutting power, and it is limited to two named accounts.** Because fifty robots stopping dead across a town's pavements is itself a hazard, and a remotely triggerable hard stop is an attack surface. Controlled stopping and a small, logged set of senders matter more than the instinct that "stop everything now" should be one click for anyone.

4.11 Try it

Do these on paper, with boxes, labelled arrows and numbers.

- 1. Warehouse to 1,000 robots.** Redo the bandwidth arithmetic in 4.3 for a thousand robots. Does the summary approach still fit on the network? What about event clips if each robot has one event an hour? Where is the first thing that breaks?
- 2. Drone inspection fleet.** Ten drones inspect wind turbines on a hill with no 4G, only a base station van with Wi-Fi at the bottom. Draw robot, fleet server, operator. Which boxes move into the van? What does store-and-forward look like when a drone is out of range for twenty minutes and its battery lasts twenty-five?

3. **Traffic on your own grid.** Draw a 6 by 6 grid with two aisles crossing. Put three robots on it with start and goal cells. Build the reservation table by hand for priority order 1, 2, 3. Then redraw the grid with one-way lanes and count the extra cells each robot drives. Which do you choose, and why, in one level-1 sentence?
 4. **Staged rollout gone wrong.** Version 85 reaches stage 2 (10 robots). Task time is fine. Near-misses go from 2 per hour to 5 per hour across those ten. Write down, in order, what the server does automatically, what the operator sees, and what a human must decide.
 5. **Hazard row.** Add two rows to the hazard table in 4.8 for the warehouse fleet, not the pavement robot: one hazard involving a person on foot in an aisle, one involving two robots. Fill in every column, including "Left over".
-

4.12 In your own words

A fleet is many complete robots plus one new system around them: a server that assigns tasks, routes traffic, owns the map and stages updates, and a person who handles exceptions. The network between them is slow, shared and often absent, so robots send small summaries, keep everything needed for a safe stop on board, and buffer what they cannot send. Traffic is planning with time added; assignment is matching that is usually greedy for speed. Updates go out in stages with a rollback ready. A safety-rated stop lives on its own circuit, and no network command can reach it. Everything else is the same four words as level 1: components, interfaces, data flow, trade-offs, just with more boxes.

4.13 Check yourself

Answer each in two or three sentences, without looking back. If one is hard, reread that section.

1. Name the five things that change when you go from one robot to two hundred, and say which box handles each.
2. What is the one rule that decides whether a job runs on the robot or in the cloud? Give one job on each side and the reason.
3. One raw camera is about 221 Mbit/s and a summary is about 200 bytes a second. Roughly what is the ratio, and what does that imply for what the fleet server sees?
4. What is a broker, what is a topic, and what is a "last will" in MQTT for?
5. A robot loses Wi-Fi for ten minutes. What does it do with its data, what does it do with its task, and what does it send first when the link returns?
6. Explain the reservation table in one paragraph. What changes about A* to use it?
7. In the aisle example, why did nearest-first cost 8 when the best was 6? What cheap fix recovers most of the gap?
8. Why does the map have a version number, and what does the server do when a robot is on an older version than the master?
9. Describe a staged rollout, what "clean" means at each stage, and why A/B partitions make rollback safe.
10. Why must the stop circuit be separate from the main computer, and why is an open network emergency stop dangerous even though it sounds safe?